Hands-On Activity 8 | Visualizing Variability

Name : Mendoza, Ryo Shin P.
Course Code and Title : CPE031 - Visualization and Data Analysis
Date Submitted : September 19, 2026
Instructor : Engr. Jimlord Quejado

1. Objectives:

This activity aims to develop students' ability to visualize deviations in data, and analyze variability with sample statistics and estimates.

2. Intended Learning Outcomes (ILOs):

By the end of this module the students should be able to:

1. Analyze how charts display represent categorical and quantitative data distribution.
2. Evaluate feature distribution and statistical measures of location and variability.
3. Create clear and coherent visualizations of sample statistics and prediction estimates.

3. Discussions:

Deviations in data values can appear within a single variable, such as the ages of Olympic gymnasts. It explains different chart types for showing how a variable's values are distributed. The choice of visualization depends on factors like whether the data is categorical or quantitative, the dataset size, and the number of variables being compared. To better interpret these visualizations, the chapter introduces basic statistical measures of central tendency and variability. It concludes with methods for visually representing variability in sample statistics and prediction estimates.

4. Procedures:

Creating Distributions from Data

In data analysis, understanding how variables vary is essential for making informed decisions. Distributions help engineers and analysts identify patterns, detect anomalies, and model real-world systems. A random variable represents a quantity whose values are uncertain, while realizations are the past observed values collected through sampling. Since it is rarely practical to analyze an entire population, analysts work with representative samples to generalize findings. Visualization techniques like histograms, frequency polygons, and relative frequency plots are critical tools for revealing the structure and variability of data.

Core Concepts

1. Random Variables and Variation
    - Random variables represent uncertain values affected by external factors.
    - Variation occurs when a variable's values differ across observations, such as over time or across customers.
    - Descriptive analytics uses data visualization to understand these variations and their impact.
2. Types of Data -.Categorical Data: Consists of labels or names that cannot be mathematically manipulated (e.g., beverage types, product categories).
    - Quantitative Data: Numeric values that can be mathematically analyzed (e.g., sales numbers, temperatures).
3. Frequency Distributions
    - Show how often specific values or ranges of values occur.
    - Categorical Example: Soft drink sales distribution to compare popularity.
    - Quantitative Example: Age distribution using ranges (bins) to capture continuous data.
4. Relative and Percent Frequency
    - Relative frequency expresses counts as proportions of the total.
    - Percent frequency provides percentages, useful for estimating probabilities.
    - Example: Using purchase data to predict required stock for a concert.

5. Benford's Law
    - Observes that in many natural datasets, lower digits (like 1) appear more frequently as the first digit.
    - Used in fraud detection and anomaly detection, especially in financial datasets.

Visualization Techniques

1. Histograms
    - Best for quantitative data.
    - Columns without spaces represent continuous ranges of values.
    - The number of bins and bin width strongly affect clarity:
    - Too many bins make data noisy.
    - Too few bins oversimplify patterns.
2. Frequency Polygons
    - Use lines instead of bars, allowing easier comparison between multiple variables.
    - Example: Comparing age distributions of male and female individuals.
3. Skewness
    - Left-skewed: Most values are high, few are low (e.g., exam scores).
    - Right-skewed: Most values are low, few are high (e.g., income data).
    - Symmetric: Balanced distribution, like heights or standardized test scores.

Key Steps for Building Distributions

1. Define Bins: Decide the number and range of bins (typically 5–20).
2. Calculate Frequencies: Use software functions like COUNTIF or FREQUENCY in Excel.
3. Visualize:
    - Clustered column charts for categorical data.
    - Histograms or frequency polygons for quantitative data.
4. Interpret Patterns: Identify trends, anomalies, or potential risks.

Coding Procedure

```
#Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Seed for reproducibility
np.random.seed(42)
```

```
# Setup
# ========================================
# Generate sample quantitative data
# ========================================
# Simulated data: 200 random ages between 15 and 70
ages = np.random.randint(15, 70, 200)

# Define bin ranges for age groups
bins = [15, 25, 35, 45, 55, 65, 75]

# ========================================
# Part 1: Core Concepts
# ========================================
# Calculate frequency and relative frequency
frequency, bin_edges = np.histogram(ages, bins=bins)
relative_frequency = frequency / len(ages)

# Create a frequency table
distribution_df = pd.DataFrame({
    "Age Range": [f"{bins[i]}-{bins[i+1]-1}" for i in range(len(bins)-1)],
    "Frequency": frequency,
    "Relative Frequency": relative_frequency.round(3)
})

print("=== Part 1: Core Concepts ===")
print("Frequency and Relative Frequency Table:")
print(distribution_df)

# ========================================
# Part 2: Visualization Techniques
# ========================================
```

```
# Calculate midpoints for the frequency polygon
midpoints = 0.5 * (bin_edges[1:] + bin_edges[:-1])

plt.figure(figsize=(10, 5))

# Histogram
plt.hist(ages, bins=bins, color='skyblue', edgecolor='black', alpha=0.7, label='Histogram')

# Frequency Polygon
plt.plot(midpoints, frequency, color='red', marker='o', linewidth=2, label='Frequency Polygon')

print("=== Part 2: Visualization Techniques ===")
plt.title("Part 2: Visualization Techniques - Age Distribution")
plt.xlabel("Age Range")
plt.ylabel("Frequency")
plt.xticks(bins)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()

# ======================================
# Part 3: Key Steps for Building Distributions
# ======================================

print("\n=== Part 3: Key Steps for Building Distributions ===")

# Step 1: Define bins
print(f"Step 1: Bins defined as {bins}")

# Step 2: Calculate frequency and relative frequency
frequency, bin_edges = np.histogram(ages, bins=bins)
relative_frequency = frequency / len(ages)

step2_df = pd.DataFrame({
    "Age Range": [f"{bins[i]}-{bins[i+1]-1}" for i in range(len(bins)-1)],
    "Frequency": frequency,
    "Relative Frequency": relative_frequency.round(3)
})

print("\nStep 2: Frequency Table")
print(step2_df)

# Step 3: Visualize the data
plt.figure(figsize=(10, 5))
plt.hist(ages, bins=bins, color='lightgreen', edgecolor='black', alpha=0.7, label='Histogram')
plt.plot(midpoints, frequency, color='blue', marker='o', linewidth=2, label='Frequency Polygon')

plt.title("Step 3: Visualization of Age Distribution")
plt.xlabel("Age Range")
plt.ylabel("Frequency")
plt.xticks(bins)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()

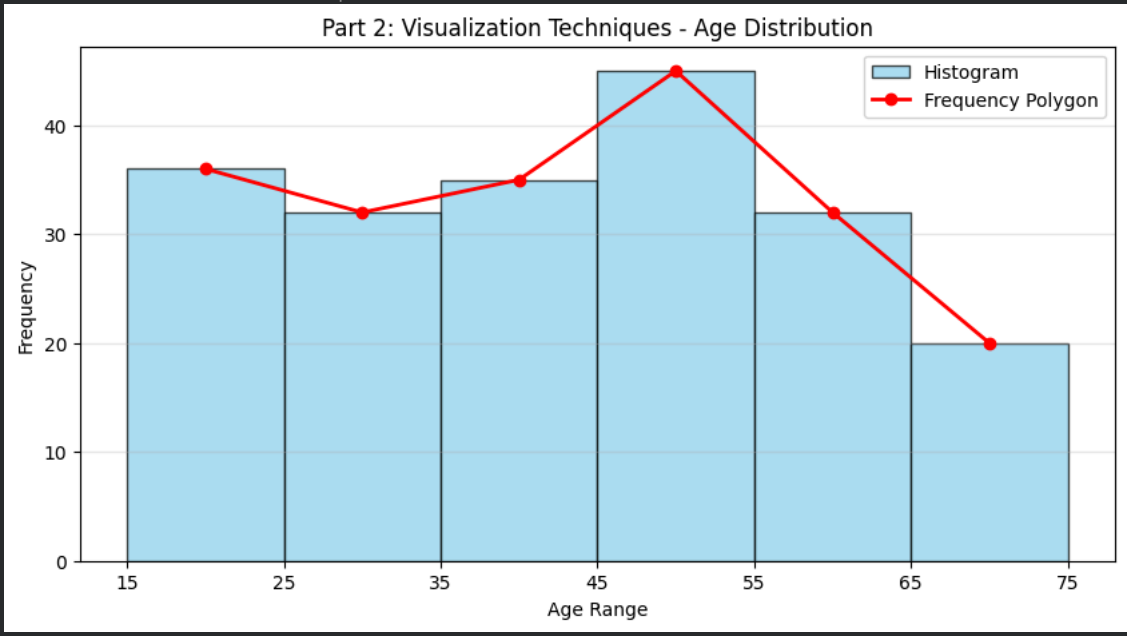
# Step 4: Interpret Patterns
max_freq_index = np.argmax(frequency)
most_common_range = step2_df.iloc[max_freq_index]["Age Range"]

print("\nStep 4: Interpretation")
print(f"- The most common age range is {most_common_range} with {frequency[max_freq_index]} individuals.")

# Check skewness
lower_half = sum(frequency[:len(frequency)//2])
upper_half = sum(frequency[len(frequency)//2:])

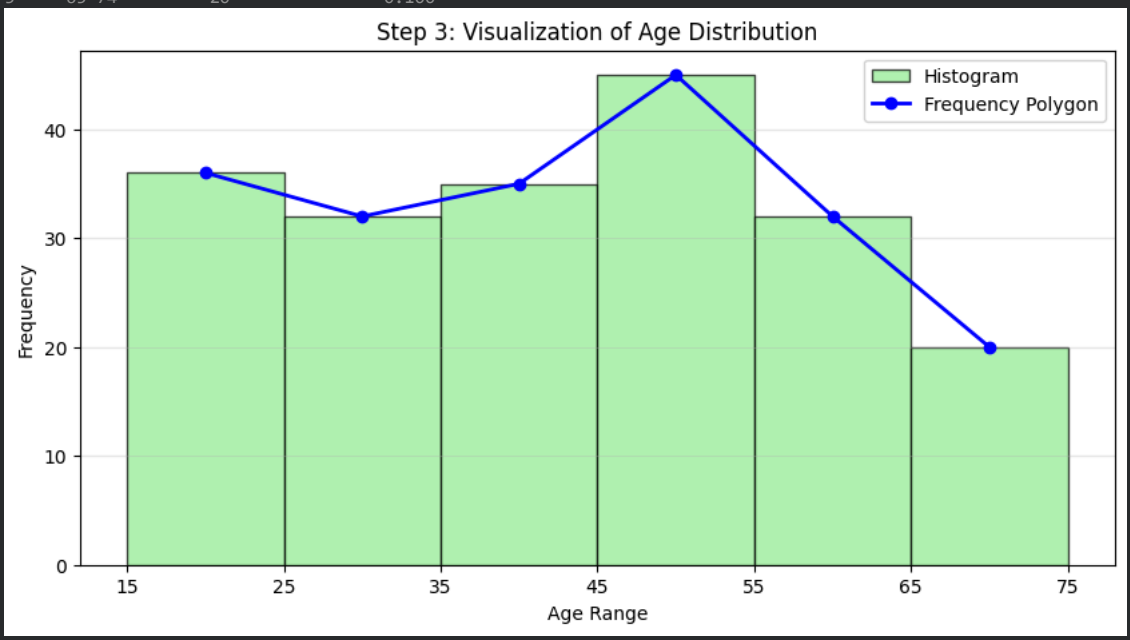
if lower_half < upper_half:
    print("- The data is LEFT-SKEWED (more older individuals).")
elif lower_half > upper_half:
    print("- The data is RIGHT-SKEWED (more younger individuals).")
else:
    print("- The data is approximately symmetric.")
```

```
=== Part 1: Core Concepts ===
Frequency and Relative Frequency Table:
  Age Range  Frequency  Relative Frequency
0     15-24         36               0.180
1     25-34         32               0.160
2     35-44         35               0.175
3     45-54         45               0.225
4     55-64         32               0.160
5     65-74         20               0.100
=== Part 2: Visualization Techniques ===
```




```
=== Part 3: Key Steps for Building Distributions ===
Step 1: Bins defined as [15, 25, 35, 45, 55, 65, 75]

Step 2: Frequency Table
  Age Range  Frequency  Relative Frequency
0     15-24         36               0.180
1     25-34         32               0.160
2     35-44         35               0.175
3     45-54         45               0.225
4     55-64         32               0.160
5     65-74         20               0.100
```




```
Step 4: Interpretation
- The most common age range is 45-54 with 45 individuals.
- The data is RIGHT-SKEWED (more younger individuals).
```

Task 1: Creating Distributions from Data

Modify the provided code to see how **bin size** affects the shape of the distribution.

Instructions:

1. Update the `bins` variable to use **different bin groupings**, such as:

- `[15, 30, 45, 60, 75]`
- `[15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75]` (many narrow bins)

2. Run the code after each change and observe how the **histogram** and **frequency polygon** change.
3. Add labels or a note in the print statements describing which bin setting is being used.

```
# Code Here
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Seed for reproducibility
np.random.seed(42)

# ======================================
# Generate sample quantitative data
# ======================================
# Simulated data: 200 random ages between 15 and 70
ages = np.random.randint(15, 70, 200)

# ======================================
# Task 1: Different Bin Sizes
# ======================================

# Change the bins here to test different groupings
bins = [15, 30, 45, 60, 75]

print("=== Task 1: Creating Distributions from Data ===")
print(f"Using bin setting: {bins}")

# Calculate frequency and relative frequency
frequency, bin_edges = np.histogram(ages, bins=bins)
relative_frequency = frequency / len(ages)

# Create a frequency table
distribution_df = pd.DataFrame({
    "Age Range": [f"{bins[i]}-{bins[i+1]-1}" for i in range(len(bins)-1)],
    "Frequency": frequency,
    "Relative Frequency": relative_frequency.round(3)
})

print("\nFrequency and Relative Frequency Table:")
print(distribution_df)

# Calculate midpoints for the frequency polygon
midpoints = 0.5 * (bin_edges[1:] + bin_edges[:-1])

# ======================================
# Visualization
# ======================================

plt.figure(figsize=(10, 5))

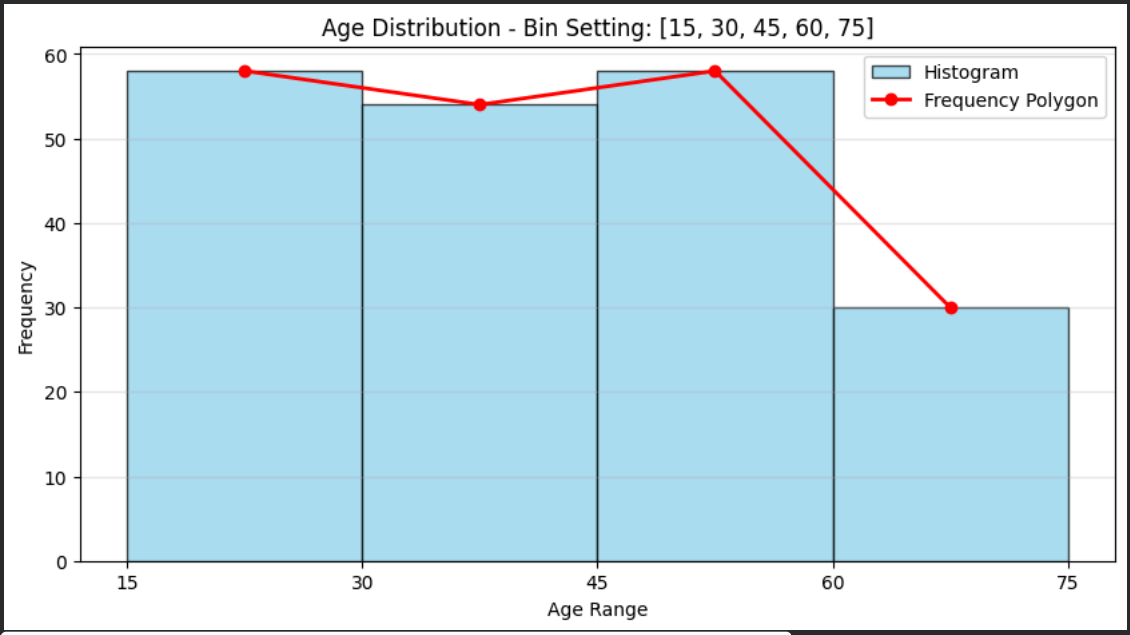
# Histogram
plt.hist(
    ages,
    bins=bins,
    color='skyblue',
    edgecolor='black',
    alpha=0.7,
    label='Histogram'
)

# Frequency Polygon
plt.plot(
    midpoints,
    frequency,
    color='red',
    marker='o',
    linewidth=2,
    label='Frequency Polygon'
)

plt.title(f"Age Distribution - Bin Setting: {bins}")
plt.xlabel("Age Range")
plt.ylabel("Frequency")
plt.xticks(bins)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()
```

```
=== Task 1: Creating Distributions from Data ===
Using bin setting: [15, 30, 45, 60, 75]

Frequency and Relative Frequency Table:
  Age Range  Frequency  Relative Frequency
0     15-29         58                0.29
1     30-44         54                0.27
2     45-59         58                0.29
3     60-74         30                0.15
```




```
# Define bin ranges for many narrow age groups
bins = [15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75]

print("=== Task 1: Many Narrow Bins ===")
print(f"Using bin setting: {bins}")

# Calculate frequency and relative frequency
frequency, bin_edges = np.histogram(ages, bins=bins)
relative_frequency = frequency / len(ages)

# Calculate midpoints
midpoints = 0.5 * (bin_edges[1:] + bin_edges[:-1])

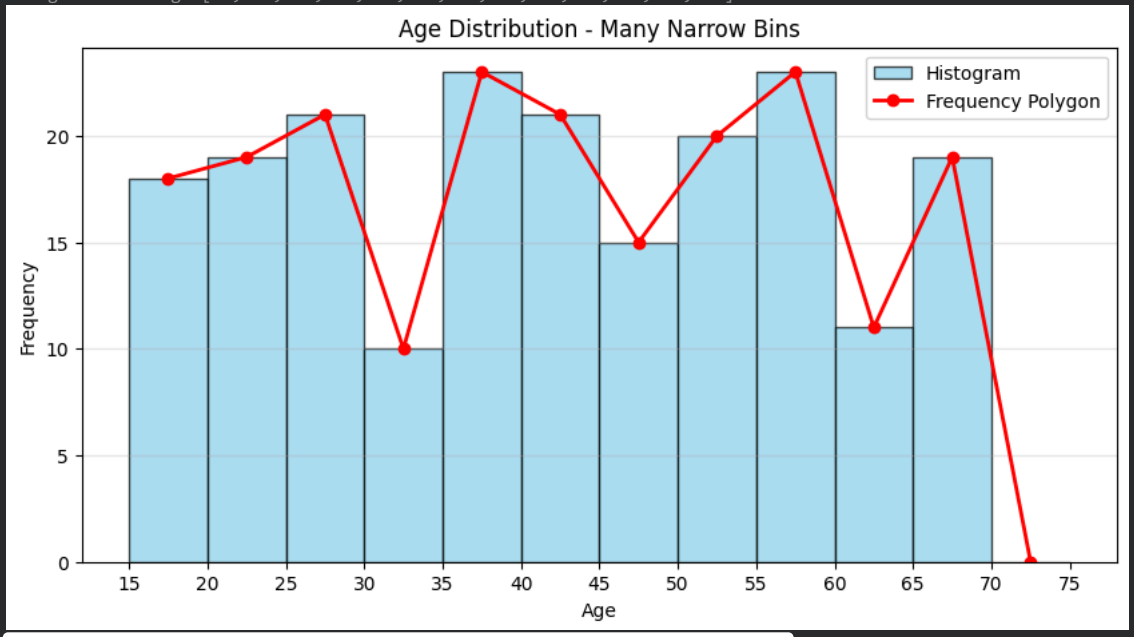
# Histogram
plt.figure(figsize=(10, 5))

plt.hist(
    ages,
    bins=bins,
    color='skyblue',
    edgecolor='black',
    alpha=0.7,
    label='Histogram'
)

# Frequency Polygon
plt.plot(
    midpoints,
    frequency,
    color='red',
    marker='o',
    linewidth=2,
    label='Frequency Polygon'
)

plt.title("Age Distribution - Many Narrow Bins")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.xticks(bins)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()
```

```
=== Task 1: Many Narrow Bins ===
Using bin setting: [15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75]
```



QUESTION: How does increasing the number of bins affect the smoothness or detail of the visualization?

Answer: Increasing the number of bins makes the visualization more detailed because it shows smaller groups of data. However, the graph can also look less smooth and more uneven because small changes in frequency become more noticeable.

Statistical Analysis of Distributions of Quantitative Variables

Understanding data distributions is essential for engineers and analysts working with real-world datasets. Statistical measures and visualization techniques help summarize, analyze, and interpret data patterns effectively. This section focuses on methods for identifying central tendencies, variability, and outliers in quantitative variables. These concepts are foundational for data-driven decision-making, predictive modeling, and system optimization in computer engineering applications.

Measures of Location (Central Tendency):

- Identify the "center" of a dataset, which represents a typical value.
- Mean (Average):
  - Sensitive to extreme values or outliers.
  - Useful when data is evenly distributed.
- Median (Middle Value):
  - Preferred when data contains extreme values or is skewed.
  - More robust than the mean for smaller datasets.
- Mode (Most Frequent Value):
  - Best for datasets with a limited range of distinct values.
  - In continuous data, represents the range with the highest frequency in a histogram.

Measures of Variability (Dispersion):

- Quantify how spread out the data values are.
- Range:
  - Simple to compute but overly affected by outliers.
- Standard Deviation:
  - Indicates average deviation from the mean.
  - Best for symmetric, bell-shaped distributions.
- Percentiles and Interquartile Range (IQR):
  - Describe data spread without being distorted by extreme values.
  - Useful for skewed datasets.
  - IQR highlights the middle 50% of data values.

Visualization Techniques: Box-and-Whisker Plots

- Combine statistical measures into a clear visual summary.
- Show median, quartiles, range, and potential outliers.
- Useful for comparing multiple datasets, such as performance metrics across different systems.
- Outliers are easily identifiable, helping engineers detect anomalies in system behavior or sensor data.

Applications in Engineering

- Identifying network latency anomalies using skewness and outlier detection.
- Analyzing processor performance data through measures of variability.
- Monitoring sensor outputs in IoT systems with box plots for quality control.
- Selecting appropriate summary measures for predictive models and machine learning pipelines.

Statistical analysis of quantitative data involves understanding central tendency, variability, and outlier behavior. Combining numerical measures with effective visualizations like box plots enables engineers to interpret data more accurately. By applying these concepts, computer engineers can improve decision-making in areas such as performance optimization, anomaly detection, and predictive analytics.

```
# ================================================
# Statistical Analysis of Distributions of Quantitative Variables
# ================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Generate sample quantitative data: 200 random ages between 15 and 70
ages = np.random.randint(15, 70, 200)

# ------------------------------------------------
# PART 1: CORE CONCEPTS - Measures of Central Tendency and Variability
# ------------------------------------------------

print("=== PART 1: CORE CONCEPTS ===")

# Calculate measures of central tendency
mean_age = np.mean(ages)
median_age = np.median(ages)
mode_age = pd.Series(ages).mode()[0]  # mode() returns a Series

# Calculate measures of variability
range_age = np.max(ages) - np.min(ages)
std_dev_age = np.std(ages)
q1 = np.percentile(ages, 25)
q3 = np.percentile(ages, 75)
iqr = q3 - q1

# Display results
print(f"Mean Age: {mean_age:.2f}")
print(f"Median Age: {median_age}")
print(f"Mode Age: {mode_age}")
print(f"Range: {range_age}")
print(f"Standard Deviation: {std_dev_age:.2f}")
print(f"Interquartile Range (IQR): {iqr}")

# Interpretation example
if std_dev_age > 10:
    print("- The data has high variability.")
else:
    print("- The data has low variability.")

# ------------------------------------------------
# PART 2: VISUALIZATION TECHNIQUES - Box Plot and Histogram
# ------------------------------------------------

print("\n=== PART 2: VISUALIZATION TECHNIQUES ===")

plt.figure(figsize=(12, 5))

# Box plot to visualize spread and outliers
plt.subplot(1, 2, 1)
plt.boxplot(ages, vert=True, patch_artist=True, boxprops=dict(facecolor='lightblue'))
plt.title("Box-and-Whisker Plot of Ages")
plt.ylabel("Age")

# Histogram to show frequency distribution
plt.subplot(1, 2, 2)
plt.hist(ages, bins=10, color='skyblue', edgecolor='black')
plt.title("Histogram of Ages")
plt.xlabel("Age")
```

```
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# ------------------------------------------------
# PART 3: KEY STEPS FOR BUILDING DISTRIBUTIONS
# ------------------------------------------------

print("\n=== PART 3: KEY STEPS FOR BUILDING DISTRIBUTIONS ===")

# Step 1: Define bins for age ranges
bins = [15, 25, 35, 45, 55, 65, 75]

# Step 2: Calculate frequencies and relative frequencies
frequency, bin_edges = np.histogram(ages, bins=bins)
relative_frequency = frequency / len(ages)

# Step 3: Create a frequency table
distribution_df = pd.DataFrame({
    "Age Range": [f"{bins[i]}-{bins[i+1]-1}" for i in range(len(bins)-1)],
    "Frequency": frequency,
    "Relative Frequency": relative_frequency.round(3)
})

print("Frequency Distribution Table:")
print(distribution_df)

# Step 4: Visualize frequency polygon with histogram
midpoints = 0.5 * (bin_edges[1:] + bin_edges[:-1])

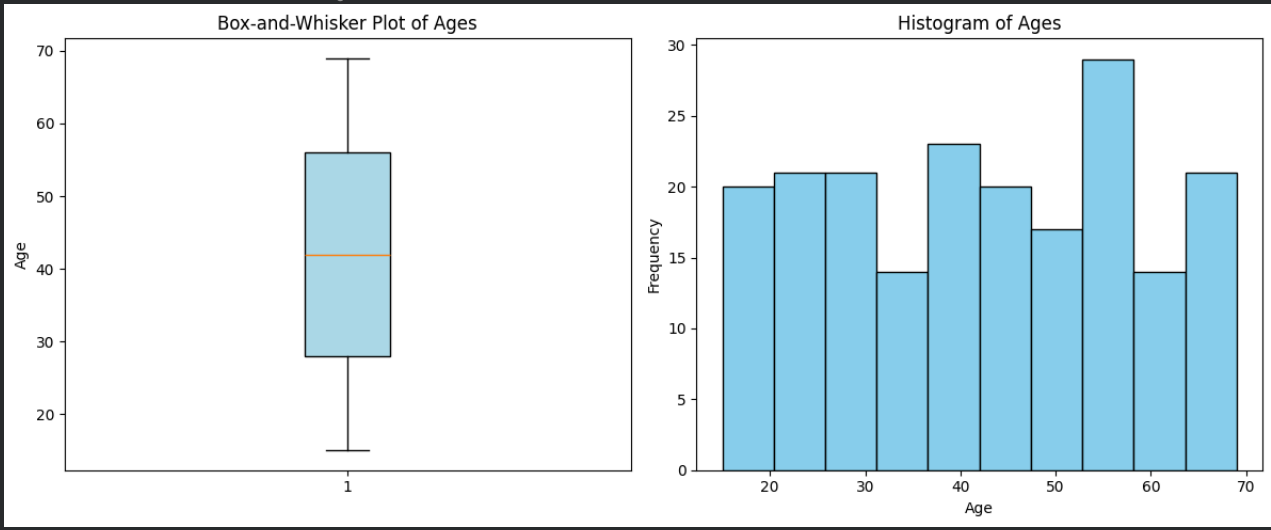
plt.figure(figsize=(8, 5))
plt.hist(ages, bins=bins, color='lightgray', edgecolor='black', alpha=0.7, label='Histogram')
plt.plot(midpoints, frequency, color='red', marker='o', linewidth=2, label='Frequency Polygon')
plt.title("Age Distribution with Frequency Polygon")
plt.xlabel("Age Range")
plt.ylabel("Frequency")
plt.xticks(bins)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()

# Interpretation: Skewness
lower_half = np.sum(ages < median_age)
upper_half = np.sum(ages > median_age)

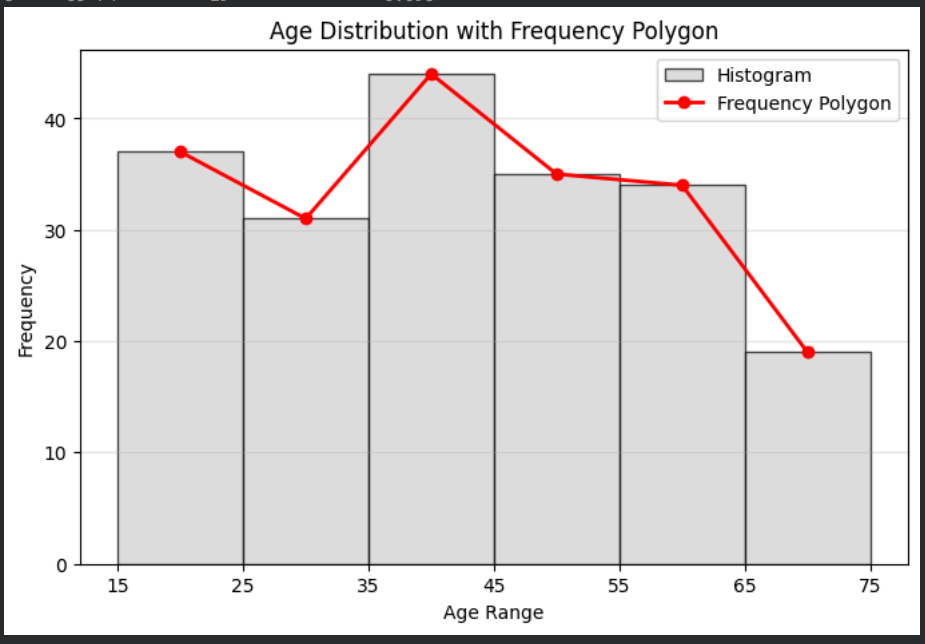
print("\nSkewness Interpretation:")
if lower_half < upper_half:
    print("- The data is LEFT-SKEWED (more older individuals, tail on the left).")
elif lower_half > upper_half:
    print("- The data is RIGHT-SKEWED (more younger individuals, tail on the right).")
else:
    print("- The data is approximately symmetric.")
```

```
=== PART 1: CORE CONCEPTS ===
Mean Age: 41.88
Median Age: 42.0
Mode Age: 58
Range: 54
Standard Deviation: 15.77
Interquartile Range (IQR): 28.0
- The data has high variability.

=== PART 2: VISUALIZATION TECHNIQUES ===
```




```
=== PART 3: KEY STEPS FOR BUILDING DISTRIBUTIONS ===
Frequency Distribution Table:
  Age Range  Frequency  Relative Frequency
0     15-24         37               0.185
1     25-34         31               0.155
2     35-44         44               0.220
3     45-54         35               0.175
4     55-64         34               0.170
5     65-74         19               0.095
```




```
Skewness Interpretation:
- The data is RIGHT-SKEWED (more younger individuals, tail on the right).
```

Task 2: Statistical Analysis of Distributions of Quantitative Variables

Modify the provided code to explore how data distribution and visualization methods affect interpretation.

Part 1 (Core Concepts):

- Change the randomly generated data to simulate different scenarios, such as network latency or response times.
- Compare how the mean and median behave when outliers are introduced.

- Add at least one extreme value manually (e.g., very high age or latency).

Part 2 (Visualization Techniques):

- Adjust the box plot color scheme to make outliers more visible.
- Experiment with the number of bins in the histogram (try 5, 10, and 15) and observe how it changes the view of the data.

Part 3 (Building Distributions):

- Modify the bin ranges to group values differently, then observe how the frequency table and frequency polygon are affected.
- Add a title and axis labels that match the new dataset you created.

```
# ================================================
# Statistical Analysis of Distributions of Quantitative Variables
# ================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Seed for reproducibility
np.random.seed(42)

# Generate sample quantitative data: 200 network latency values
# Most values are between 20 and 100 milliseconds
latency = np.random.normal(60, 15, 200)

# Add one extreme value manually
latency = np.append(latency, 300)

# ------------------------------------------------
# PART 1: CORE CONCEPTS - Measures of Central Tendency and Variability
# ------------------------------------------------

print("=== PART 1: CORE CONCEPTS ===")

# Calculate measures of central tendency
mean_latency = np.mean(latency)
median_latency = np.median(latency)
mode_latency = pd.Series(latency.round(2)).mode()[0]

# Calculate measures of variability
range_latency = np.max(latency) - np.min(latency)
std_dev_latency = np.std(latency)
q1 = np.percentile(latency, 25)
q3 = np.percentile(latency, 75)
iqr = q3 - q1

# Display results
print(f"Mean Latency: {mean_latency:.2f} ms")
print(f"Median Latency: {median_latency:.2f} ms")
print(f"Mode Latency: {mode_latency:.2f} ms")
print(f"Range: {range_latency:.2f} ms")
print(f"Standard Deviation: {std_dev_latency:.2f} ms")
print(f"Interquartile Range (IQR): {iqr:.2f} ms")

# Interpretation
if mean_latency > median_latency:
    print("- The extreme value increased the mean more than the median.")
else:
    print("- The mean and median are close in value.")

# ------------------------------------------------
# PART 2: VISUALIZATION TECHNIQUES
# ------------------------------------------------

print("\n=== PART 2: VISUALIZATION TECHNIQUES ===")

# Box Plot
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.boxplot(
    latency,
    vert=True,
    patch_artist=True,
    boxprops=dict(facecolor='lightgreen'),
    flierprops=dict(
        marker='o',
        markerfacecolor='red',
```

```
        markeredgecolor='black',
        markersize=8
    )
)

plt.title("Box Plot of Network Latency")
plt.ylabel("Latency (ms)")

# Histogram with 10 bins
plt.subplot(1, 2, 2)

plt.hist(
    latency,
    bins=10,
    color='skyblue',
    edgecolor='black'
)

plt.title("Histogram of Network Latency")
plt.xlabel("Latency (ms)")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# ------------------------------------------------
# PART 2: Histogram with Different Number of Bins
# ------------------------------------------------

print("\n=== HISTOGRAM BIN COMPARISON ===")

# 5 bins
plt.figure(figsize=(8, 5))
plt.hist(latency, bins=5, color='skyblue', edgecolor='black')
plt.title("Network Latency - 5 Bins")
plt.xlabel("Latency (ms)")
plt.ylabel("Frequency")
plt.show()

# 10 bins
plt.figure(figsize=(8, 5))
plt.hist(latency, bins=10, color='lightgreen', edgecolor='black')
plt.title("Network Latency - 10 Bins")
plt.xlabel("Latency (ms)")
plt.ylabel("Frequency")
plt.show()

# 15 bins
plt.figure(figsize=(8, 5))
plt.hist(latency, bins=15, color='lightcoral', edgecolor='black')
plt.title("Network Latency - 15 Bins")
plt.xlabel("Latency (ms)")
plt.ylabel("Frequency")
plt.show()

# ------------------------------------------------
# PART 3: KEY STEPS FOR BUILDING DISTRIBUTIONS
# ------------------------------------------------

print("\n=== PART 3: KEY STEPS FOR BUILDING DISTRIBUTIONS ===")

# Step 1: Define bins for network latency
bins = [0, 30, 50, 70, 90, 110, 150, 300]

print(f"Step 1: Bins defined as {bins}")

# Step 2: Calculate frequencies and relative frequencies
frequency, bin_edges = np.histogram(latency, bins=bins)
relative_frequency = frequency / len(latency)

# Step 3: Create a frequency table
distribution_df = pd.DataFrame({
    "Latency Range (ms)": [
        f"{bins[i]}-{bins[i+1]-1}"
        for i in range(len(bins)-1)
    ],
    "Frequency": frequency,
    "Relative Frequency": relative_frequency.round(3)
})

print("\nStep 2: Frequency Distribution Table:")
print(distribution_df)
```

```
# Step 4: Visualize frequency polygon with histogram
midpoints = 0.5 * (bin_edges[1:] + bin_edges[:-1])

plt.figure(figsize=(10, 5))

plt.hist(
    latency,
    bins=bins,
    color='lightgray',
    edgecolor='black',
    alpha=0.7,
    label='Histogram'
)

plt.plot(
    midpoints,
    frequency,
    color='red',
    marker='o',
    linewidth=2,
    label='Frequency Polygon'
)

plt.title("Network Latency Distribution with Frequency Polygon")
plt.xlabel("Latency (ms)")
plt.ylabel("Frequency")
plt.xticks(bins)
plt.legend()
plt.grid(axis='y', alpha=0.3)

plt.show()

# ------------------------------------------------
# Interpretation
# ------------------------------------------------

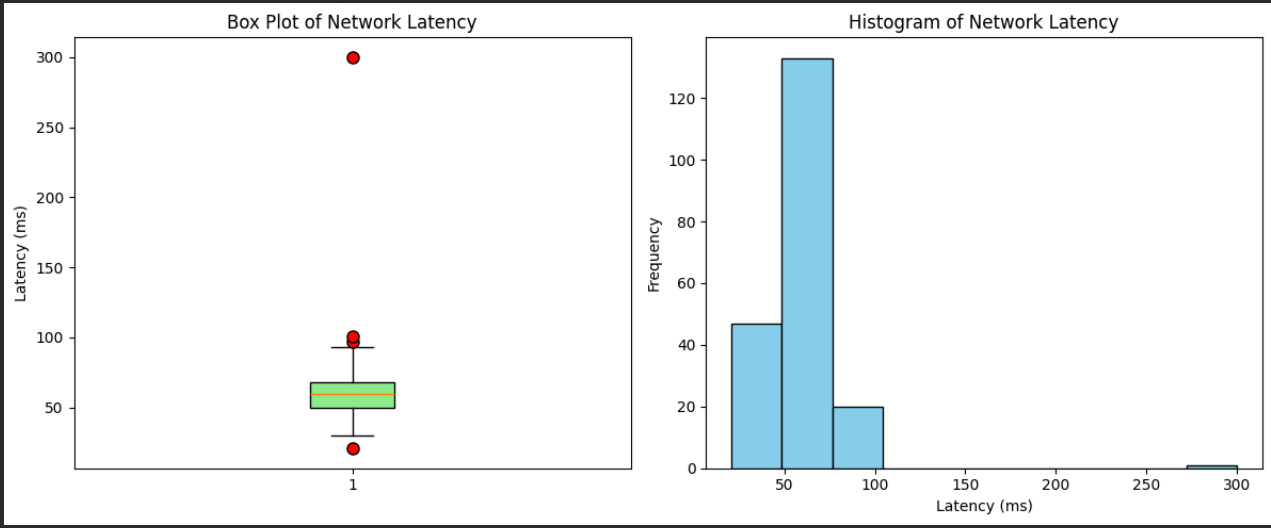
print("\n=== INTERPRETATION ===")

if mean_latency > median_latency:
    print("- The mean is higher than the median because of the extreme latency value.")
    print("- The distribution is right-skewed because of the very high latency.")

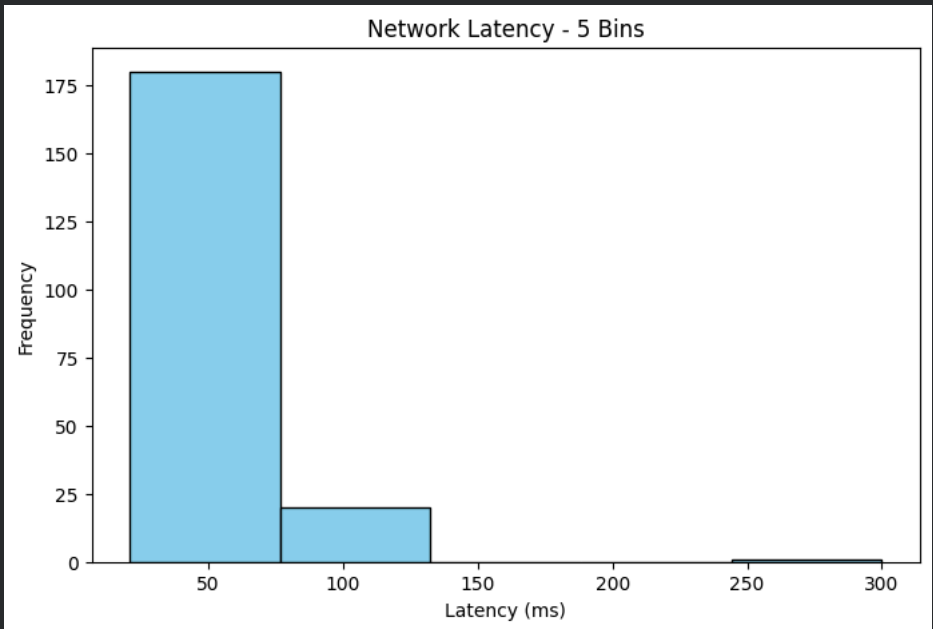
print("- The box plot makes the extreme value easier to identify.")
print("- Fewer bins give a simpler view of the data.")
print("- More bins show more details and changes in the distribution.")
```

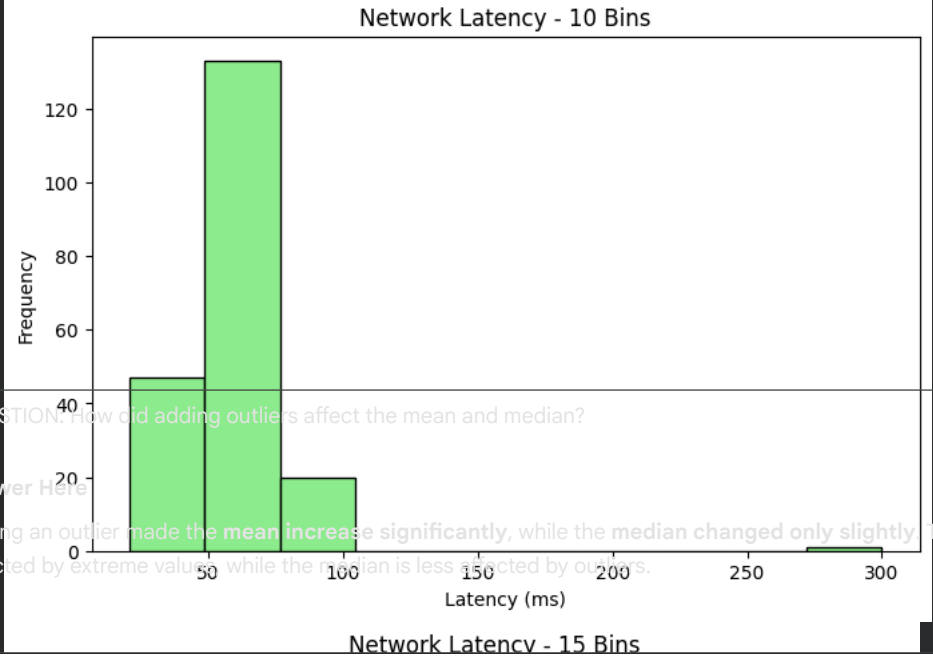
```
=== PART 1: CORE CONCEPTS ===
Mean Latency: 60.59 ms
Median Latency: 60.08 ms
Mode Latency: 56.49 ms
Range: 279.30 ms
Standard Deviation: 21.90 ms
Interquartile Range (IQR): 18.23 ms
- The extreme value increased the mean more than the median.

=== PART 2: VISUALIZATION TECHNIQUES ===
```




```
=== HISTOGRAM BIN COMPARISON ===
```





QUESTION: How did adding outliers affect the mean and median?

Answer Here

Adding an outlier made the **mean increase significantly**, while the **median changed only slightly**. This is because the mean is more affected by extreme values, while the median is less affected by outliers.

Network Latency - 15 Bins

Uncertainty in Sample Statistics

In data analysis, we often rely on samples to estimate characteristics of a larger population. However, because a sample is only a subset of the entire population, there is always uncertainty in how accurately sample results reflect the true population values. This uncertainty must be clearly communicated through statistical measures and visualizations to avoid misleading conclusions. By using tools like confidence intervals and visual aids such as error bars, analysts can provide a clearer picture of the reliability of their findings. These techniques are essential for engineers and data scientists when making decisions based on limited data, such as system monitoring, user behavior analysis, or predictive modeling.

Statistical Inference and Sampling

- Statistical inference is the process of using a sample to draw conclusions about a population.
- Uncertainty arises because samples are incomplete representations of the population.
- Common real-world applications include political polling, market research, and quality control.

Confidence Intervals (CIs)

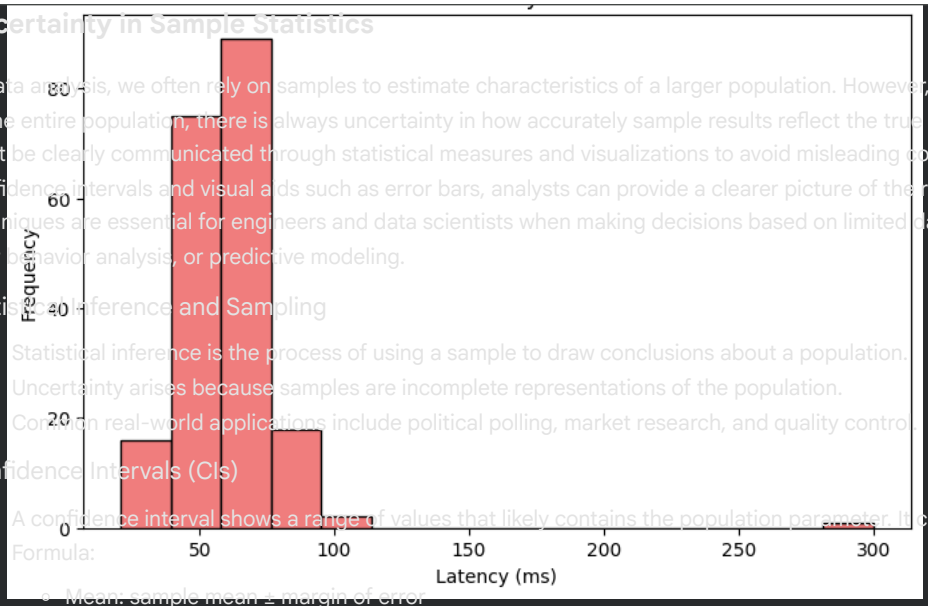
- A confidence interval shows a range of values that likely contains the population parameter. It can be showed in two variations:
- Formula:
  - Mean: sample mean ± margin of error
  - Proportion: sample proportion ± margin of error
- Influencing factors:
  - Sample size: Larger samples reduce uncertainty and narrow the interval.
  - Data variability: Greater variability increases uncertainty and widens the interval.
  - Confidence level: Higher confidence levels (e.g., 95% or 99%) require wider intervals.

Visualization Techniques

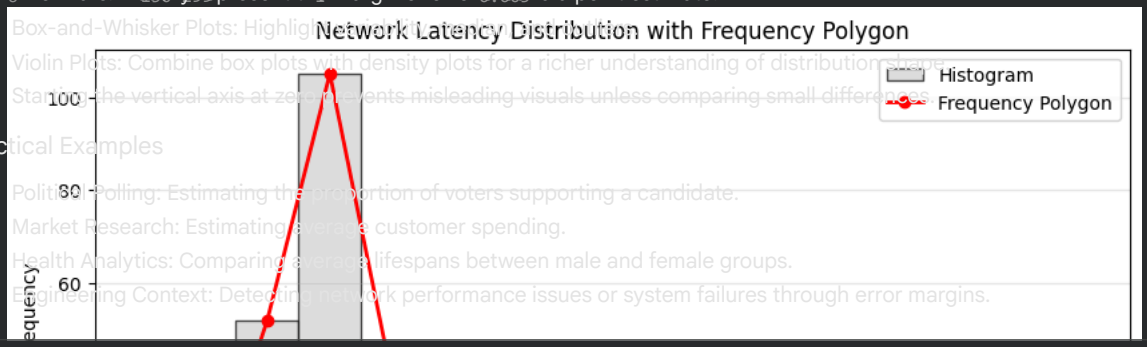
- Error Bars: Visually represent the margin of error around a point estimate.
- Box-and-Whisker Plots: Highlight the median, quartiles, and outliers of data.
- Violin Plots: Combine box plots with density plots for a richer understanding of distribution shapes.
- Start the vertical axis at zero to avoid misleading visuals unless comparing small differences.

Practical Examples

- Political Polling: Estimating the proportion of voters supporting a candidate.
- Market Research: Estimating average customer spending.
- Health Analytics: Comparing average lifespans between male and female groups.
- Engineering Context: Detecting network performance issues or system failures through error margins.




```
=== PART 1: Frequency Distribution Table ===
Step 1: Bins defined as [0, 30, 50, 70, 90, 110, 150, 300]
Step 2: Frequency Distribution Table:
  Latency Range (ms)  Frequency  Relative Frequency
0              0-29          1               0.005
1             30-49         52               0.259
2             50-69        105               0.522
3             70-89         39               0.194
4             90-109         3               0.015
5           110-149         0               0.000
6           150-299         1               0.005
```




```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

# ==============================
# PART 1: Statistical Inference and Sampling
# ==============================
print("=== Part 1: Statistical Inference and Sampling ===")

# Population data: Simulate a population of 10,000 network latency values (in ms)
population = np.random.normal(loc=100, scale=15, size=10000)  # mean=100ms, std=15ms

# Draw a random sample of 50
sample_size = 50
sample = np.random.choice(population, sample_size, replace=False)

# Calculate population and sample means
population_mean = np.mean(population)
sample_mean = np.mean(sample)

print(f"Population Mean (True): {population_mean:.2f} ms")
print(f"Sample Mean (Estimate): {sample_mean:.2f} ms")
print(f"Sample Size: {sample_size}", end="\n\n")

# ==============================
# PART 2: Confidence Intervals
# ==============================
print("=== Part 2: Confidence Intervals ===")

# Calculate 95% confidence interval for the sample mean
confidence_level = 0.95
std_error = np.std(sample, ddof=1) / np.sqrt(sample_size)
```

```
margin_of_error = stats.t.ppf((1 + confidence_level) / 2, df=sample_size-1) * std_error

ci_lower = sample_mean - margin_of_error
ci_upper = sample_mean + margin_of_error

print(f"95% Confidence Interval: [{ci_lower:.2f}, {ci_upper:.2f}] ms")

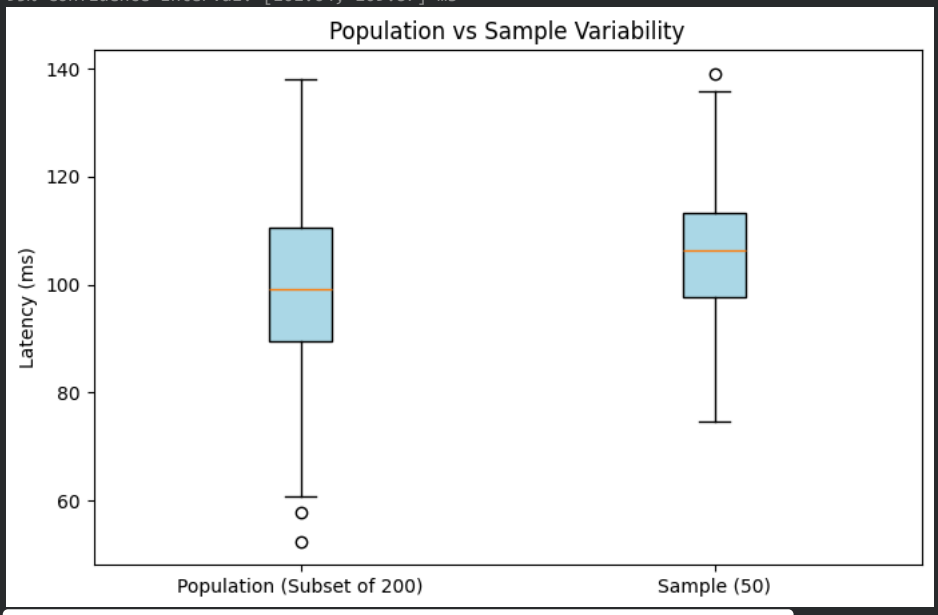
# ==============================
# PART 3: Visualization Techniques
# ==============================

# Create a dataframe for plotting
population_subset = np.random.choice(population, 200, replace=False)  # 200 for visualization

# Boxplot directly from separate arrays
plt.figure(figsize=(8, 5))
plt.boxplot(
    [population_subset, sample],
    tick_labels=['Population (Subset of 200)', 'Sample (50)'],
    patch_artist=True,
    boxprops=dict(facecolor='lightblue')
)
plt.title("Population vs Sample Variability")
plt.ylabel("Latency (ms)")
plt.show()
```

```
=== Part 1: Statistical Inference and Sampling ===
Population Mean (True): 99.99 ms
Sample Mean (Estimate): 105.76 ms
Sample Size: 50

=== Part 2: Confidence Intervals ===
95% Confidence Interval: [101.64, 109.87] ms
```



Task 3: Uncertainty in Sample Statistics

Modify the provided code to explore how sampling decisions and visualization methods affect uncertainty in statistical analysis.

Part 1: Sampling

- Change the sample size from 50 to 30 and 100.
- Compare how the sample mean differs from the population mean for each sample size.
- Print all sample means alongside the true population mean.

Part 2: Confidence Intervals

- Compute and print the 95% confidence intervals for each sample size.
- Observe how the width of the interval changes when the sample size increases or decreases.
- Highlight which sample size provides the most reliable estimate.

Part 3: Visualization

- Adjust the boxplot to include all three sample sets side by side along with the population subset.
- Add a clear title and proper axis labels.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

# ==============================
# PART 1: Statistical Inference and Sampling
# ==============================
print("=== Part 1: Statistical Inference and Sampling ===")

# Population data: 10,000 network latency values
population = np.random.normal(loc=100, scale=15, size=10000)

# Calculate the true population mean
population_mean = np.mean(population)

# Define different sample sizes
sample_sizes = [30, 50, 100]

# Store samples and sample means
samples = {}
sample_means = {}

# Draw samples and calculate means
for size in sample_sizes:
    samples[size] = np.random.choice(
        population,
        size,
        replace=False
    )

    sample_means[size] = np.mean(samples[size])

# Print population mean
print(f"Population Mean (True): {population_mean:.2f} ms")

# Print all sample means
for size in sample_sizes:
    print(f"Sample Mean (n={size}): {sample_means[size]:.2f} ms")

print()


# ==============================
# PART 2: Confidence Intervals
# ==============================
print("=== Part 2: Confidence Intervals ===")

confidence_level = 0.95

for size in sample_sizes:

    sample = samples[size]
    sample_mean = sample_means[size]

    # Calculate standard error
    std_error = np.std(sample, ddof=1) / np.sqrt(size)

    # Calculate margin of error
    margin_of_error = stats.t.ppf(
        (1 + confidence_level) / 2,
        df=size - 1
    ) * std_error

    # Calculate confidence interval
    ci_lower = sample_mean - margin_of_error
    ci_upper = sample_mean + margin_of_error

    # Calculate interval width
    interval_width = ci_upper - ci_lower

    print(f"\nSample Size: {size}")
    print(f"Sample Mean: {sample_mean:.2f} ms")
    print(f"95% Confidence Interval: [{ci_lower:.2f}, {ci_upper:.2f}] ms")
    print(f"Interval Width: {interval_width:.2f} ms")

print("\nInterpretation:")
print("- A larger sample size produces a narrower confidence interval.")
print("- A smaller sample size produces a wider confidence interval.")
print("- The sample size of 100 provides the most reliable estimate because it has less uncertainty.")
```

```
# ==============================
# PART 3: Visualization Techniques
# ==============================
print("\n=== Part 3: Visualization ===")

# Population subset for visualization
population_subset = np.random.choice(
    population,
    200,
    replace=False
)

# Create boxplot
plt.figure(figsize=(10, 6))

plt.boxplot(
    [
        population_subset,
        samples[30],
        samples[50],
        samples[100]
    ],
    tick_labels=[
        'Population\n(200)',
        'Sample\n(n=30)',
        'Sample\n(n=50)',
        'Sample\n(n=100)'
    ],
    patch_artist=True,
    boxprops=dict(facecolor='lightblue')
)

plt.title("Population and Sample Variability by Sample Size")
plt.xlabel("Population and Sample Groups")
plt.ylabel("Network Latency (ms)")

plt.grid(axis='y', alpha=0.3)

plt.show()
```

```
=== Part 1: Statistical Inference and Sampling ===
Population Mean (True): 99.97 ms
Sample Mean (n=30): 99.01 ms
Sample Mean (n=50): 102.69 ms
Sample Mean (n=100): 97.50 ms

=== Part 2: Confidence Intervals ===

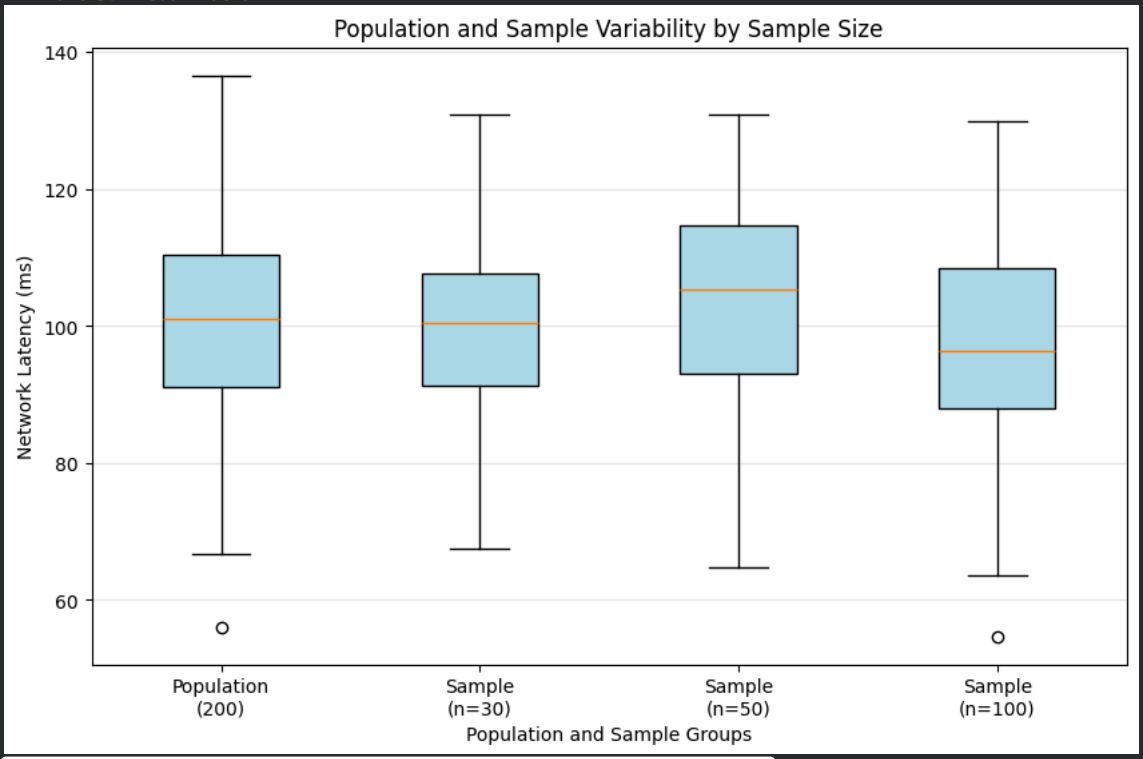
Sample Size: 30
Sample Mean: 99.01 ms
95% Confidence Interval: [94.49, 103.53] ms
Interval Width: 9.04 ms

Sample Size: 50
Sample Mean: 102.69 ms
95% Confidence Interval: [98.16, 107.22] ms
Interval Width: 9.06 ms

Sample Size: 100
Sample Mean: 97.50 ms
95% Confidence Interval: [94.49, 100.52] ms
Interval Width: 6.03 ms

Interpretation:
- A larger sample size produces a narrower confidence interval.
- A smaller sample size produces a wider confidence interval.
- The sample size of 100 provides the most reliable estimate because it has less uncertainty.

=== Part 3: Visualization ===
```



QUESTION: How did the sample size affect the accuracy of the sample mean compared to the population mean?

Answer Here

As the sample size increased, the **sample mean generally became closer to the population mean**. Smaller samples had more variation, while larger samples gave a **more accurate and stable estimate** of the population mean.

Uncertainty in Predictive Models

Predictive analytics uses past data to create models that forecast future outcomes. These models are widely applied in engineering and business, such as predicting network latency, product sales, or travel times. However, predictions are never exact, and uncertainty must be considered. Visualization of this uncertainty helps engineers and decision-makers understand the reliability of forecasts, improving real-world decision-making.

Predictive Models and Point Estimates

- Predictive models generate point estimates, which are single predicted values for future observations.
- These estimates contain error because real-world systems involve randomness and variability.

Prediction Intervals

- A prediction interval gives a range where future outcomes are likely to occur with a certain confidence level (e.g., 95%).
- Similar to confidence intervals used in statistical inference, but focused on predicting individual future observations rather than population parameters.

Simple Linear Regression Example

- Linear regression models relationships between two variables:
  - Independent variable (x): factor used for prediction (e.g., number of requests in a delivery route).
  - Dependent variable (y): outcome being predicted (e.g., travel time).
- Visualization includes:
  - Historical data points.
  - Regression line showing predicted values.
  - Upper and lower dashed lines showing prediction interval boundaries.
- Key observation: intervals are narrower near the average values of the independent variable and wider at extremes, indicating higher uncertainty for unusual cases.

Time Series Models Example

- Used for data measured sequentially over time (e.g., quarterly sales).
- Predictions account for patterns like seasonality or trends.
- Visualization shows:
  - Past data as a continuous line.
  - Future point predictions as a lighter line.
  - Prediction intervals widening as the forecast extends further into the future.
- Important insight: uncertainty grows over time because long-term predictions are inherently less reliable.

```
# ================================================
# Uncertainty in Predictive Models
# ================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression

# ================================================
# PART 1: Predictive Models and Point Estimates
# ================================================
print("=== Part 1: Predictive Models and Point Estimates ===")

# Generate random data: relationship between requests and travel time
num_requests = np.random.randint(1, 10, size=10)  # independent variable (random integers 1-9)
travel_time = np.random.normal(loc=3, scale=0.5, size=10)  # dependent variable with random noise

# Fit a simple linear regression model
X = num_requests.reshape(-1, 1)
y = travel_time
model = LinearRegression().fit(X, y)

# Predict point estimates
predictions = model.predict(X)

print("Regression Model Summary:")
print(f"Intercept: {model.intercept_:.3f}, Slope: {model.coef_[0]:.3f}\n")

print("Sample of Predictions:")
for i, p in zip(num_requests, predictions):
    print(f"Requests: {i}, Predicted Travel Time: {p:.2f} hours")

# ================================================
# PART 2: Prediction Intervals
# ================================================
print("\n=== Part 2: Prediction Intervals ===")

# Use statsmodels to compute prediction intervals
X_with_const = sm.add_constant(X)  # statsmodels requires intercept column
ols_model = sm.OLS(y, X_with_const).fit()

# Generate prediction intervals
pred_summary = ols_model.get_prediction(X_with_const).summary_frame(alpha=0.05)  # 95% PI
lower_bounds = pred_summary["obs_ci_lower"]
upper_bounds = pred_summary["obs_ci_upper"]

# Display prediction intervals
```

```
print(pred_summary[["mean", "obs_ci_lower", "obs_ci_upper"]])

# ================================================
# PART 3: Visualization for Linear Regression
# ================================================
print("\n=== Part 3: Visualization for Linear Regression ===")

# Sort data for a cleaner plot
sorted_indices = np.argsort(num_requests)
sorted_requests = num_requests[sorted_indices]
sorted_predictions = predictions[sorted_indices]
sorted_lower = lower_bounds[sorted_indices]
sorted_upper = upper_bounds[sorted_indices]
sorted_actual = travel_time[sorted_indices]

plt.figure(figsize=(8, 6))
plt.scatter(sorted_requests, sorted_actual, color='blue', label='Observed Data')
plt.plot(sorted_requests, sorted_predictions, color='orange', label='Regression Line')
plt.fill_between(sorted_requests, sorted_lower, sorted_upper,
                 color='orange', alpha=0.3, label='95% Prediction Interval')
plt.xlabel("Number of Requests")
plt.ylabel("Travel Time (hours)")
plt.title("Linear Regression with Prediction Intervals")
plt.legend()
plt.show()

# ================================================
# PART 4: Time Series Forecasting with Prediction Intervals
# ================================================
print("\n=== Part 4: Time Series Forecasting ===")

# Generate random time series data for quarterly sales
time_periods = 17
quarters = np.arange(1, time_periods + 1)
sales = np.random.normal(loc=150, scale=20, size=time_periods)  # random sales data

# Create dataframe
df_sales = pd.DataFrame({"Quarter": quarters, "Sales": sales})

# Simulate future predictions
future_periods = 8
future_quarters = np.arange(time_periods + 1, time_periods + future_periods + 1)

# Random predictions with added seasonality-like variation
future_sales_pred = np.random.normal(loc=150, scale=15, size=future_periods)

# Add growing uncertainty to future predictions
future_uncertainty = np.linspace(10, 30, future_periods)
lower_future = future_sales_pred - future_uncertainty
upper_future = future_sales_pred + future_uncertainty

# Combine into dataframe
forecast_df = pd.DataFrame({
    "Quarter": future_quarters,
    "Predicted_Sales": future_sales_pred,
    "Lower_PI": lower_future,
    "Upper_PI": upper_future
})

# ================================================
# Visualization for Time Series
# ================================================
plt.figure(figsize=(10, 6))
plt.plot(df_sales["Quarter"], df_sales["Sales"], color='darkblue', label='Historical Sales')
plt.plot(forecast_df["Quarter"], forecast_df["Predicted_Sales"], color='lightblue', label='Forecast')
plt.fill_between(forecast_df["Quarter"], forecast_df["Lower_PI"], forecast_df["Upper_PI"],
                 color='lightblue', alpha=0.3, label='95% Prediction Interval')
plt.xlabel("Quarter")
plt.ylabel("Sales ($1000s)")
plt.title("Time Series Forecast with Prediction Intervals")
plt.legend()
plt.show()
```

```
=== Part 1: Predictive Models and Point Estimates ===
Regression Model Summary:
Intercept: 2.989, Slope: 0.034

Sample of Predictions:
Requests: 1, Predicted Travel Time: 3.02 hours
Requests: 8, Predicted Travel Time: 3.26 hours
Requests: 2, Predicted Travel Time: 3.06 hours
Requests: 8, Predicted Travel Time: 3.26 hours
Requests: 7, Predicted Travel Time: 3.23 hours
Requests: 2, Predicted Travel Time: 3.06 hours
Requests: 6, Predicted Travel Time: 3.19 hours
Requests: 8, Predicted Travel Time: 3.26 hours
Requests: 2, Predicted Travel Time: 3.06 hours
Requests: 2, Predicted Travel Time: 3.06 hours
=== Part 2: Prediction Intervals ===
      mean  mean_ci_lower  mean_ci_upper
0  3.022463       1.918040       4.126885
1  3.259269       2.154847       4.363692
2  3.056292       1.990363       4.122221
3  3.259269       2.154847       4.363692
4  3.225540       2.150511       4.301369
5  3.056292       1.990363       4.122221
6  3.191818       2.115115       4.268521
7  3.191818       2.152115       4.231521
8  3.090122       2.050647       4.129597
9  3.056292       1.990363       4.122221
```

Task 4: Uncertainty in Predictive Models

Modify the provided code to explore how uncertainty changes under different scenarios. Complete the following steps:

Part 1: Predictive Models and Point Estimates

- Increase the sample size of the random data by generating 30 data points instead of 10.
- Observe how the regression model changes with more data.

Part 2: Prediction Intervals

- Change the confidence level from 95% to 80% when generating prediction intervals.
- Compare how the width of the intervals changes.

Part 3: Visualization

- Change the scatter plot color for observed data to green.
- Increase the transparency (alpha) of the shaded prediction interval region to 0.5 for better visibility.
- Modify the plot title to: "Effect of Sample Size and Confidence Level on Prediction Intervals".

```
# ================================================
# Uncertainty in Predictive Models
# ================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression

# ================================================
# PART 1: Predictive Models and Point Estimates
# ================================================
print("=== Part 1: Predictive Models and Point Estimates ===")

# Generate random data: relationship between requests and travel time
# Increased sample size from 10 to 30 data points
num_requests = np.random.randint(1, 10, size=30)
travel_time = np.random.normal(loc=3, scale=0.5, size=30)

# Fit a simple linear regression model
X = num_requests.reshape(-1, 1)
y = travel_time
model = LinearRegression().fit(X, y)

# Predict point estimates
predictions = model.predict(X)

print("Regression Model Summary:")
print(f"Intercept: {model.intercept_:.3f}, Slope: {model.coef_[0]:.3f}\n")

print("Sample of Predictions:")
for i, p in zip(num_requests, predictions):
    print(f"Requests: {i}, Predicted Travel Time: {p:.2f} hours")


# ================================================
# PART 2: Prediction Intervals
# ================================================
print("\n=== Part 2: Prediction Intervals ===")

# Use statsmodels to compute prediction intervals
X_with_const = sm.add_constant(X)
ols_model = sm.OLS(y, X_with_const).fit()

# Change confidence level from 95% to 80%
confidence_level = 0.80
alpha = 1 - confidence_level

# Generate 80% prediction intervals
pred_summary = ols_model.get_prediction(X_with_const).summary_frame(alpha=alpha)

lower_bounds = pred_summary["obs_ci_lower"]
upper_bounds = pred_summary["obs_ci_upper"]

# Display prediction intervals
print("80% Prediction Intervals:")
```

```
print(pred_summary[["mean", "obs_ci_lower", "obs_ci_upper"]])


# ============================================
# PART 3: Visualization for Linear Regression
# ============================================
print("\n=== Part 3: Visualization for Linear Regression ===")

# Sort data for a cleaner plot
sorted_indices = np.argsort(num_requests)

sorted_requests = num_requests[sorted_indices]
sorted_predictions = predictions[sorted_indices]
sorted_lower = lower_bounds[sorted_indices]
sorted_upper = upper_bounds[sorted_indices]
sorted_actual = travel_time[sorted_indices]

plt.figure(figsize=(8, 6))

# Observed data changed to green
plt.scatter(
    sorted_requests,
    sorted_actual,
    color='green',
    label='Observed Data'
)

# Regression line
plt.plot(
    sorted_requests,
    sorted_predictions,
    color='orange',
    label='Regression Line'
)

# 80% Prediction Interval
# Alpha changed from 0.3 to 0.5
plt.fill_between(
    sorted_requests,
    sorted_lower,
    sorted_upper,
    color='orange',
    alpha=0.5,
    label='80% Prediction Interval'
)

plt.xlabel("Number of Requests")
plt.ylabel("Travel Time (hours)")

# Updated title
plt.title("Effect of Sample Size and Confidence Level on Prediction Intervals")

plt.legend()
plt.show()


# ============================================
# PART 4: Time Series Forecasting with Prediction Intervals
# ============================================
print("\n=== Part 4: Time Series Forecasting ===")

# Generate random time series data for quarterly sales
time_periods = 17
quarters = np.arange(1, time_periods + 1)
sales = np.random.normal(loc=150, scale=20, size=time_periods)

# Create dataframe
df_sales = pd.DataFrame({
    "Quarter": quarters,
    "Sales": sales
})

# Simulate future predictions
future_periods = 8
future_quarters = np.arange(
    time_periods + 1,
    time_periods + future_periods + 1
)

# Random predictions with added variation
future_sales_pred = np.random.normal(
    loc=150,
```

```
        scale=15,
        size=future_periods
    )

    # Add growing uncertainty to future predictions
    future_uncertainty = np.linspace(10, 30, future_periods)

    lower_future = future_sales_pred - future_uncertainty
    upper_future = future_sales_pred + future_uncertainty

    # Combine into dataframe
    forecast_df = pd.DataFrame({
        "Quarter": future_quarters,
        "Predicted_Sales": future_sales_pred,
        "Lower_PI": lower_future,
        "Upper_PI": upper_future
    })

    # ============================================
    # Visualization for Time Series
    # ============================================
    plt.figure(figsize=(10, 6))

    plt.plot(
        df_sales["Quarter"],
        df_sales["Sales"],
        color='darkblue',
        label='Historical Sales'
    )

    plt.plot(
        forecast_df["Quarter"],
        forecast_df["Predicted_Sales"],
        color='lightblue',
        label='Forecast'
    )

    plt.fill_between(
        forecast_df["Quarter"],
        forecast_df["Lower_PI"],
        forecast_df["Upper_PI"],
        color='lightblue',
        alpha=0.3,
        label='95% Prediction Interval'
    )

    plt.xlabel("Quarter")
    plt.ylabel("Sales ($1000s)")
    plt.title("Time Series Forecast with Prediction Intervals")
    plt.legend()

    plt.show()
```

```
=== Part 1: Predictive Models and Point Estimates ===
Regression Model Summary:
Intercept: 3.063, Slope: 0.006

Sample of Predictions:
Requests: 5, Predicted Travel Time: 3.10 hours
Requests: 6, Predicted Travel Time: 3.10 hours
Requests: 7, Predicted Travel Time: 3.11 hours
Requests: 9, Predicted Travel Time: 3.12 hours
Requests: 7, Predicted Travel Time: 3.11 hours
Requests: 1, Predicted Travel Time: 3.07 hours
Requests: 5, Predicted Travel Time: 3.10 hours
Requests: 4, Predicted Travel Time: 3.09 hours
Requests: 8, Predicted Travel Time: 3.11 hours
Requests: 7, Predicted Travel Time: 3.11 hours
Requests: 4, Predicted Travel Time: 3.09 hours
Requests: 3, Predicted Travel Time: 3.08 hours
Requests: 6, Predicted Travel Time: 3.10 hours
Requests: 8, Predicted Travel Time: 3.11 hours
Requests: 3, Predicted Travel Time: 3.08 hours
Requests: 5, Predicted Travel Time: 3.10 hours
Requests: 1, Predicted Travel Time: 3.07 hours
Requests: 8, Predicted Travel Time: 3.11 hours
Requests: 7, Predicted Travel Time: 3.11 hours
Requests: 8, Predicted Travel Time: 3.11 hours
Requests: 2, Predicted Travel Time: 3.08 hours
Requests: 7, Predicted Travel Time: 3.11 hours
Requests: 4, Predicted Travel Time: 3.09 hours
Requests: 2, Predicted Travel Time: 3.08 hours
Requests: 9, Predicted Travel Time: 3.12 hours
Requests: 5, Predicted Travel Time: 3.10 hours
Requests: 7, Predicted Travel Time: 3.11 hours
Requests: 4, Predicted Travel Time: 3.09 hours
Requests: 4, Predicted Travel Time: 3.09 hours
Requests: 3, Predicted Travel Time: 3.08 hours

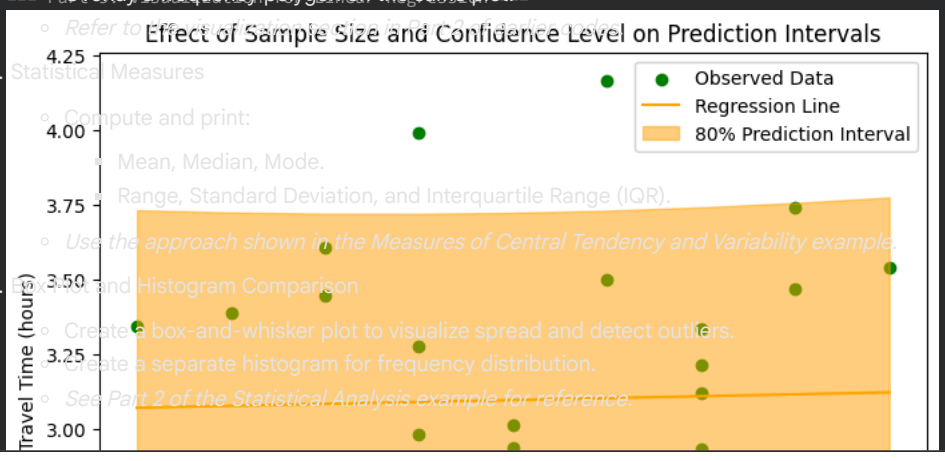
=== Part 2: Prediction Intervals ===
80% Prediction Intervals:
        mean  obs_ci_lower  obs_ci_upper
0   3.095023      2.467930      3.722116
1   3.101455      2.473596      3.729314
2   3.107887      2.475453      3.740321
3   3.120751      2.468123      3.773378
4   3.107887      2.475453      3.740321
5   3.069296      2.407892      3.730700
6   3.095023      2.467930      3.722116
7   3.088591      2.458441      3.718742
8   3.114319      2.473581      3.755057
9   3.107887      2.475453      3.740321
10  3.088591      2.458441      3.718742
11  3.082160      2.445183      3.719136
12  3.101455      2.473596      3.729314
13  3.114319      2.473581      3.755057
14  3.082160      2.445183      3.719136
15  3.095023      2.467930      3.722116
16  3.069296      2.407892      3.730700
17  3.114319      2.473581      3.755057
18  3.107887      2.475452      3.740321
19  3.114319      2.473581      3.755057
20  3.075728      2.428275      3.723180
21  3.107887      2.475453      3.740321
22  3.088591      2.458441      3.718742
23  3.075728      2.428275      3.723180
24  3.120751      2.468123      3.773378
25  3.095023      2.467930      3.722116
26  3.107887      2.475453      3.740321
27  3.088591      2.458441      3.718742
28  3.088591      2.458441      3.718742
29  3.082160      2.445183      3.719136

=== Part 3: Residual Analysis and Model Diagnostics ===
```

5. Supplementary Activity:

Demonstrate the key steps for building, analyzing, and visualizing data distributions.

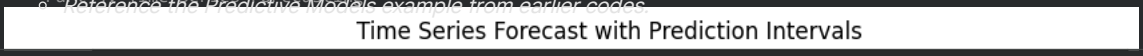
1. Create a synthetic dataset
   - Generate one dataset of 200 random ages between 15 and 70. This dataset will be used throughout all steps.
2. Core Concepts: Frequency and Relative Frequency
   - Build a frequency table showing:
     - Age ranges divided into bins of 10 years (15-24, 25-34, etc.).
     - The frequency and relative frequency of each bin.
   - *Reference the frequency table example shown in earlier parts.*
3. Visualization Techniques
   - Create a histogram to display the age distribution.
   - Overlay a frequency polygon on the same plot.
   - *Refer to the visualization techniques in Part 2 of earlier modules.*
4. Statistical Measures
   - Compute and print:
     - Mean, Median, Mode.
     - Range, Standard Deviation, and Interquartile Range (IQR).
   - *Use the approach shown in the Measures of Central Tendency and Variability example.*
5. Boxplot and Histogram Comparison
   - Create a box-and-whisker plot to visualize spread and detect outliers.
   - Create a separate histogram for frequency distribution.
   - *See Part 2 of the Statistical Analysis example for reference.*



6. Prediction and Uncertainty Visualization
   - Using the same dataset, simulate a simple linear regression where:
     - x is a random subset of 10 values from ages.
     - y is a random dependent variable with noise.
   - Fit the regression and plot:
     - Observed data
     - Regression line
     - 95% prediction intervals shaded on the plot.



=== Part 4: Time Series Forecasting ===
- *Reference the Predictive Models example from earlier codes*




```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression

# ======================================================
# Step 1: Create synthetic dataset
# ======================================================
# Generate a dataset of 200 random ages between 15 and 70
np.random.seed(42)
ages = np.random.randint(15, 70, 200)

# ======================================================
# Step 2: Frequency and Relative Frequency Table
# ======================================================

# Define bin ranges for age groups
bins = [15, 25, 35, 45, 55, 65, 75]

# Calculate frequency and relative frequency
frequency, bin_edges = np.histogram(ages, bins=bins)
relative_frequency = frequency / len(ages)

# Create a DataFrame for the frequency table
distribution_df = pd.DataFrame({
    "Age Range": [
        "15-24",
        "25-34",
        "35-44",
        "45-54",
        "55-64",
        "65-74"
    ],
    "Frequency": frequency,
    "Relative Frequency": relative_frequency.round(3)
})

# Print frequency table
print("=== Frequency and Relative Frequency Table ===")
print(distribution_df)


# ======================================================
# Step 3: Visualization Techniques
# ======================================================

# Calculate midpoints for frequency polygon
midpoints = 0.5 * (bin_edges[1:] + bin_edges[:-1])

plt.figure(figsize=(10, 5))

# Create histogram of ages
plt.hist(
    ages,
    bins=bins,
    edgecolor='black',
    alpha=0.7,
    label='Histogram'
)

# Overlay frequency polygon on top of histogram
plt.plot(
    midpoints,
    frequency,
    marker='o',
    linewidth=2,
```

```
    label='Frequency Polygon'
)

plt.title("Age Distribution with Frequency Polygon")
plt.xlabel("Age Range")
plt.ylabel("Frequency")
plt.xticks(bins)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()


# ====================================================
# Step 4: Statistical Measures
# ====================================================

# Calculate mean, median, and mode
mean_age = np.mean(ages)
median_age = np.median(ages)
mode_age = pd.Series(ages).mode()[0]

# Calculate range, standard deviation, and IQR
range_age = np.max(ages) - np.min(ages)
std_dev_age = np.std(ages)

q1 = np.percentile(ages, 25)
q3 = np.percentile(ages, 75)
iqr = q3 - q1

# Print results with interpretations
print("\n=== Statistical Measures ===")
print(f"Mean: {mean_age:.2f}")
print(f"Median: {median_age:.2f}")
print(f"Mode: {mode_age}")
print(f"Range: {range_age}")
print(f"Standard Deviation: {std_dev_age:.2f}")
print(f"IQR: {iqr:.2f}")

if std_dev_age > 10:
    print("- The data has relatively high variability.")
else:
    print("- The data has relatively low variability.")


# ====================================================
# Step 5: Box Plot and Histogram
# ====================================================

plt.figure(figsize=(12, 5))

# Create box plot of ages
plt.subplot(1, 2, 1)
plt.boxplot(
    ages,
    vert=True,
    patch_artist=True,
    boxprops=dict(facecolor='lightblue')
)
plt.title("Box-and-Whisker Plot of Ages")
plt.ylabel("Age")

# Create histogram of ages (separate subplot)
plt.subplot(1, 2, 2)
plt.hist(
    ages,
    bins=10,
    edgecolor='black',
    alpha=0.7
)
plt.title("Histogram of Ages")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()


# ====================================================
# Step 6: Prediction and Uncertainty Visualization
# ====================================================

# Randomly select 10 age values as x
```

```
x = np.random.choice(ages, 10, replace=False)

# Create y values with random noise
y = np.random.normal(loc=3, scale=0.5, size=10)

# Fit linear regression model
model = LinearRegression().fit(x.reshape(-1, 1), y)

# Generate predictions
X = x.reshape(-1, 1)
predictions = model.predict(X)

# Use statsmodels to compute prediction intervals
# Prepare data for statsmodels
X_with_const = sm.add_constant(X)

# Fit OLS model
ols_model = sm.OLS(y, X_with_const).fit()

# Get prediction summary with intervals
pred_summary = ols_model.get_prediction(
    X_with_const
).summary_frame(alpha=0.05)

# Extract lower and upper bounds
lower_bounds = pred_summary["obs_ci_lower"].values
upper_bounds = pred_summary["obs_ci_upper"].values


# Visualization
plt.figure(figsize=(8, 6))

# Sort values for a cleaner plot
sorted_indices = np.argsort(x)

sorted_x = x[sorted_indices]
sorted_y = y[sorted_indices]
sorted_predictions = predictions[sorted_indices]
sorted_lower = lower_bounds[sorted_indices]
sorted_upper = upper_bounds[sorted_indices]

# Scatter plot of observed data
plt.scatter(
    sorted_x,
    sorted_y,
    label='Observed Data'
)

# Plot regression line
plt.plot(
    sorted_x,
    sorted_predictions,
    linewidth=2,
    label='Regression Line'
)

# Shade prediction interval region
plt.fill_between(
    sorted_x,
    sorted_lower,
    sorted_upper,
    alpha=0.3,
    label='95% Prediction Interval'
)

plt.xlabel("Number of Requests (simulated x)")
plt.ylabel("Simulated Outcome (y)")
plt.title("Linear Regression with Prediction Intervals")
plt.legend()
plt.show()


# ======================================================
# Step 7: Interpretation and Reflection
# ======================================================

# Identify most common age range
max_freq_index = np.argmax(frequency)
most_common_range = distribution_df.iloc[max_freq_index]["Age Range"]

print(f"\n- The most common age range is {most_common_range}.")
```

```
    # Determine if distribution is skewed
    lower_half = np.sum(ages < median_age)
```